==========
SOURCE: agent.md
==========

Godling AI Instruction Router

1. 这份文件怎么用

这份 `agent.md` 不再是“每次都完整塞给 AI 的总方案”，而是一个分层路由器。

目标：

- 保留统一的产品方向和系统边界
- 避免把整份长文档反复灌给每一次实现任务
- 让 AI 每次只拿到当前任务真正需要的最小上下文

后续分配实现任务时，不要把整个 `agent.md` 原样复制给 AI。应只给：

1. `L0` 核心约束
2. `L1` 单局主循环
3. 1 份当前任务对应的 `L2` 子系统文档
4. 只有在修改接口、JSON、状态结构时，才补 `L3` 数据契约
5. 如需任务说明模板，再补 `Task Packet`

==========

2. 分层文件清单

L0：所有任务都要带

- [docs/agent/00-core-guardrails.md](/Users/zhangwei/Documents/Mycode/Godling/docs/agent/00-core-guardrails.md)

用途：

- 产品北极星
- 不可违背的实现约束
- 顶层状态边界

L1：所有会改动玩法流程的任务都要带

- [docs/agent/10-run-loop.md](/Users/zhangwei/Documents/Mycode/Godling/docs/agent/10-run-loop.md)

用途：

- 单局搜打撤主循环
- 回合刷新逻辑
- 撤离与死亡
- 风险收益曲线

L2A：世界层 / 事件层

- [docs/agent/20-world-events.md](/Users/zhangwei/Documents/Mycode/Godling/docs/agent/20-world-events.md)

用途：

- 随机事件与固定事件
- 任务生成
- 叙事触发线
- 地图事件刷新与保留规则

L2B：战斗层

- [docs/agent/30-battle-executor.md](/Users/zhangwei/Documents/Mycode/Godling/docs/agent/30-battle-executor.md)

用途：

- 战斗作为事件执行器的边界
- 战斗输入输出
- 单位、AI、战斗内事件 MVP

L2C：背包 / 装备 / 进度层

- [docs/agent/40-inventory-progression.md](/Users/zhangwei/Documents/Mycode/Godling/docs/agent/40-inventory-progression.md)

用途：

- 局内临时战利品
- 局外永久仓库
- 藏品装备
- 剧情标记和系统解锁

L3：数据契约

- [docs/agent/50-data-contracts.md](/Users/zhangwei/Documents/Mycode/Godling/docs/agent/50-data-contracts.md)

用途：

- JSON 结构
- 运行时状态结构
- 跨系统接口

Task Packet：任务分派模板

- [docs/agent/90-task-packets.md](/Users/zhangwei/Documents/Mycode/Godling/docs/agent/90-task-packets.md)

用途：

- 给 AI 发具体实现任务时的最小上下文模板
- 规定每类任务应附带哪些层

==========

3. 正确的事件模型

这里修正一个关键理解，后续所有实现都以这里为准。

3.1 回合内不是固定“三选一”

每回合不是固定三个事件，也不是“战斗/随机/叙事各一个”。

正确规则：

- 每回合生成大约 `4-6` 个随机事件
- 同时存在不定数的固定事件
- 玩家每回合只处理其中 `1` 个事件

3.2 事件有两条维度，不要混为一谈

事件需要拆成两条独立维度：

- `spawn_mode`：事件如何出现在地图上
- `resolution_type`：事件如何被处理

`spawn_mode`：

- `random_slot`：从随机池中抽取，刷新在地图固定位置上，下一回合可能被刷新走
- `fixed_line`：来自任务线、叙事线、key 物品或保底逻辑，不会因为随机刷新而消失

`resolution_type`：

- `battle`
- `random`
- `narrative`

注意：

- 叙事事件大多数属于 `fixed_line`
- 叙事事件的核心特征是“有稳定触发线或进入条件”
- 它们不应该被普通随机刷新覆盖掉

3.3 地图上的呈现规则

- 随机事件刷新在地图固定位置上
- 固定事件数量不定，通常应单独显示或挂在固定锚点上
- 固定事件在被解决、失效或被明确替换前，默认持续存在

==========

4. 任务到分层文档的映射

工程初始化 / Autoload / JSON 导入

给 AI：

- `L0`
- `L1`
- `L3`

地图事件刷新 / 任务池 / 固定事件线

给 AI：

- `L0`
- `L1`
- `L2A`
- `L3`

战斗场景 / 单位系统 / 战斗内事件

给 AI：

- `L0`
- `L1`
- `L2B`
- `L3`

背包 / 奖励结算 / 撤离 / 存档

给 AI：

- `L0`
- `L1`
- `L2C`
- `L3`

世界地图 UI / 事件卡 / 任务面板

给 AI：

- `L0`
- `L1`
- `L2A`
- 如涉及背包页签，再补 `L2C`

数据结构调整 / 新增 JSON 字段 / 跨系统接口变更

给 AI：

- `L0`
- 相关 `L2`
- `L3`

==========

5. 上下文压缩规则

后续给 AI 下达任务时，遵守以下规则：

1. 不要把整套长文档每次重发。
2. 默认只发 `L0 + 1 份 L1 + 1 份 L2`。
3. 只有修改接口或配置结构时才加 `L3`。
4. 如果任务只涉及单个子系统，不要附带其他子系统说明。
5. 如果任务目标能用 10 句话说清，就不要再附 200 行背景。

==========

6. 当前阶段的实现目标

当前阶段仍然是做可玩的 MVP，但 AI 在执行时必须围绕“单局多回合搜打撤闭环”分块推进，而不是把战斗系统

最小闭环：

`进入地图 -> 生成 4-6 个随机事件和若干固定事件 -> 选择并处理 1 个事件 -> 事件结算 -> 回合推进

只要这条链还没稳定，就不应优先扩展复杂 AI、复杂掉落池或大型 UI。

==========
SOURCE: docs/agent/00-core-guardrails.md
==========

L0 Core Guardrails

1. 产品北极星

`Godling` 的核心不是“做一个自动战斗系统”，而是“做一个由任务驱动叙事的 2D 单局搜打撤循环”。

AI 在实现任何功能时，都必须优先服务这条主线：

- 玩家进入单局
- 每回合面对一组可选事件
- 处理一个事件并承担风险
- 决定是否继续深入
- 成功撤离则带出物资
- 死亡则失去本局大部分战利品
- 任务、剧情和系统解锁跨局保留

2. 永远优先的设计判断

如果出现取舍冲突，优先级如下：

1. 单局主循环能否闭合
2. 任务和叙事是否真正驱动玩家决策
3. 局内收益与局外成长是否清晰分层
4. 系统是否数据驱动
5. 战斗是否足够复杂

结论：

- 不能为了加厚战斗而破坏主循环节奏
- 不能为了偷快写死剧情和解锁
- 不能让临时收益和永久收益混在一起

3. 顶层状态边界

所有实现必须围绕三个顶层状态组织：

`ContentDB`

- 只负责静态内容定义
- 来源是 JSON，不是运行时 Excel
- 负责 `id` 查询、标签检索和引用校验

`ProgressionState`

- 只负责跨局永久状态
- 包含永久仓库、永久货币、已完成任务、剧情标记、系统解锁

`RunState`

- 只负责当前单局状态
- 包含当前地图、回合、危险度、事件列表、临时战利品、局内地图数值、局内完成任务

4. 不可违背的规则

事件规则

- 每回合不是固定三个事件
- 每回合应有大约 `4-6` 个随机事件
- 同时有不定数固定事件
- 玩家每回合只处理 `1` 个事件

固定事件规则

- 叙事事件大多属于固定事件
- 固定事件默认不会被随机刷新覆盖
- 固定事件只有在被完成、失效或被明确替换时才消失

收益规则

- 局内获得的物品、货币、藏品先进入临时容器
- 只有成功撤离才进入永久存档
- 死亡会清空未撤离战利品
- `story_flags` 和 `unlock_flags` 默认即时生效并永久保留

数据规则

- 新能力优先加配置，不要优先加硬编码
- 不允许多个系统各自维护重复状态
- 所有跨系统结果都要通过结构化结果对象回传

5. 开发中的负面信号

出现以下情况时，应立即收手并重构：

- 战斗系统越写越厚，但撤离、任务推进、局内结算还没通
- UI 直接写死剧情和解锁，不经过状态层
- 世界层和背包层都在各自结算奖励
- 固定事件被当成普通随机卡位刷新掉
- 叙事事件被当成“第三种随机卡片类型”强行平均分配

6. 最小成功标准

任何实现完成后，都要问：

1. 这次改动是否让单局闭环更完整？
2. 这次改动是否保留了任务驱动叙事的定位？
3. 这次改动是否遵守了临时收益和永久收益分层？
4. 这次改动是否把规则沉淀为状态和配置，而不是临时逻辑？

==========
SOURCE: docs/agent/10-run-loop.md
==========

L1 Run Loop

1. 单局流程概述

游戏的主循环不是自由探索，而是“单局内按回合推进事件”。

标准流程：

1. 进入地图
2. 初始化 `RunState`
3. 生成本回合随机事件和固定事件
4. 玩家选择并处理 `1` 个事件
5. 结算奖励、损失、任务推进和地图数值变化
6. 回合结束，危险度提升
7. 重新生成下回合随机事件并保留有效固定事件
8. 玩家决定继续或撤离
9. 撤离或死亡后生成局终结算

2. 回合内事件板规则

随机事件

- 每回合生成大约 `4-6` 个
- 它们来自随机池
- 它们刷新在地图上的固定位置
- 若本回合未被选择，下一回合可能被刷新走

固定事件

- 数量不定
- 来自任务线、叙事线、key 物品、保底触发或特殊条件
- 不占随机事件槽位
- 默认跨回合保留

玩家选择

- 玩家每回合只处理 `1` 个事件
- 被选中的事件进入结算
- 其他随机事件在回合结束后可被刷新
- 固定事件按自身规则保留、推进、失效或替换

3. 事件的两条轴

不要把“怎么出现”和“怎么处理”混成一个枚举。

轴 1：`spawn_mode`

- `random_slot`
- `fixed_line`

轴 2：`resolution_type`

- `battle`
- `random`
- `narrative`

推荐理解：

- 大多数叙事事件：`fixed_line + narrative`
- 常规战斗事件：`random_slot + battle`
- 部分特殊剧情战斗：`fixed_line + battle`
- 纯数值或物资型探索：`random_slot + random`

4. 回合详细生命周期

`TurnStart`

- 更新危险度系数的展示值
- 检查固定事件触发条件
- 挂载新出现的固定事件
- 为随机事件槽生成 `4-6` 个候选项
- 将随机事件放入地图固定位置

`PlayerChoose`

- 玩家从全部可见事件中选择一个
- 记录本回合选中的事件和位置
- 锁定本回合，避免多次处理

`Resolve`

- 若 `resolution_type = battle`，进入战斗场景
- 若 `resolution_type = random`，在地图层直接结算
- 若 `resolution_type = narrative`，进入叙事面板和选项分支

`ApplyResult`

- 写入临时战利品
- 修改地图数值
- 更新已完成任务
- 写入 `story_flags`
- 写入 `unlock_flags`
- 处理固定事件状态变化

`TurnEnd`

- `turn + 1`
- 增加 `danger_level`
- 刷新随机事件
- 保留未失效的固定事件
- 允许玩家继续或撤离

5. 撤离和死亡

撤离

- MVP 中默认必定成功
- 将本局临时物品、货币、藏品写入永久存档
- 保留本局获得的剧情标记和系统解锁
- 生成 `ExtractionResult`

死亡

- 清空本局临时物品、货币、藏品
- 保留本局获得的剧情标记和系统解锁
- 结束单局

6. 风险收益曲线

每回合推进后，应同时发生两类变化。

风险提升

- 更高危险度
- 更高强度随机战斗
- 更激进的敌对权重

收益提升

- 更高掉落期望
- 更高价值任务
- 更深层剧情和系统开放机会

如果只能感知风险，玩家会过早撤离。

如果只能感知收益，撤离决策就失去意义。

7. L1 验收标准

- 开局能生成随机事件和固定事件
- 随机事件数量在 `4-6` 区间内
- 固定事件不会被随机刷新覆盖
- 每回合只能处理 `1` 个事件
- 事件处理后能进入下一回合
- 完成至少 `1` 个事件后允许撤离
- 撤离与死亡产出不同的结算结果

==========

SOURCE: docs/agent/20-world-events.md
==========

L2A World Events

1. 本层职责

本层负责“地图上出现什么事件、为什么出现、出现后如何持续或消失”。

它不负责：

- 战斗内部的 AI、碰撞和攻击细节
- 永久仓库的保存实现

它负责：

- 任务池
- 事件池
- 随机事件刷新
- 固定事件线
- 叙事触发条件
- 保底逻辑
- key 物品触发逻辑

2. 事件板模型

地图上的可见事件应该拆成两个集合：

`random_slots`

- 固定数量的地图位置
- 每回合填入 `4-6` 个随机事件实例
- 未被选择时，下回合可以被替换

`fixed_events`

- 不定数量
- 来自任务线、剧情线、保底触发、key 物品
- 不占 `random_slots`
- 默认跨回合保留

不要把固定事件硬塞进随机槽位。

3. 推荐运行时结构

建议世界层维护以下运行时对象：

`TurnBoardState`

- `random_slots`
- `fixed_events`
- `selected_event_id`

`RandomEventInstance`

- `instance_id`
- `event_id`
- `slot_id`
- `spawn_turn`

`FixedEventInstance`

- `instance_id`
- `event_id`
- `line_id`
- `state`
- `spawn_turn`
- `expire_turn`

4. 事件来源

随机事件来源

- 常规随机池
- 按地图、危险度、剧情状态过滤
- 按权重抽取后填充随机槽位

固定事件来源

- 主线任务链
- 支线任务链
- 叙事触发事件
- 保底计数器
- key 物品触发
- 解锁后常驻的新系统入口

5. 任务与事件的关系

推荐设计:

- `TaskDef` 负责叙事包装和生成条件
- `EventDef` 负责一次具体可处理的事件实例

`TaskDef` 需要回答:

- 什么时候能出现
- 出现在什么地图
- 展示什么任务名、目标、影响、奖励预览
- 完成后推动哪些剧情和解锁

`EventDef` 需要回答:

- 它是随机出现还是固定出现
- 它是战斗、随机结算还是叙事分支
- 被选择后如何结算

6. 固定事件线

固定事件线是叙事的主要承载方式。

要求：

- 叙事事件大多数使用固定事件线
- 固定事件必须有明确进入条件
- 固定事件默认持续存在，直到完成、失效或被显式替换
- 固定事件可以进一步派生新的固定事件

推荐字段：

- `line_id`
- `entry_conditions`
- `persistence`
- `replace_on_complete`
- `expire_conditions`

7. 随机事件生成规则

推荐生成流程：

1. 收集当前地图允许的随机事件候选
2. 按危险度、地图数值、剧情状态、最近几回合历史进行过滤
3. 应用权重修正
4. 抽取 `4-6` 个事件
5. 分配到地图固定位置

建议额外规则：

- 避免连续多回合同一位置刷新完全相同事件
- 可以按地图区域控制某类事件更常出现
- 允许某些固定事件压制部分随机池

8. `EventResolver` 边界

世界层中的 `EventResolver` 负责：

- 接收选中的事件实例
- 根据 `resolution_type` 分派到不同结算路径
- 生成统一结果对象
- 将结果回写给 `RunState`

它不应负责：

- 战斗内部每一帧如何推进
- 永久存档如何落盘

9. 世界层验收标准

- 地图上能同时显示随机事件和固定事件
- 随机事件刷新在固定地图位置
- 固定事件不会被普通刷新移除
- 任务和叙事事件能通过固定事件线持续推进
- 选择一个事件后，世界层能收到标准化结算结果

==========
SOURCE: docs/agent/30-battle-executor.md
==========

L2B Battle Executor

1. 本层定位

战斗系统不是整局主控制器，而是 `resolution_type = battle` 时被调用的事件执行器。

世界层负责决定“何时进入战斗”和“战斗结果如何影响单局”。

战斗层负责：

- 加载战斗模板
- 生成单位和战斗内事件
- 运行自动战斗
- 回传结构化战斗结果

2. MVP 范围

第一阶段只做最低可玩战斗：

- 三类单位：敌人、友方、英雄
- 单位不可重叠
- 步行和飞行
- 近战和平射
- 基础追杀 AI
- 基础召唤 / buff 战斗内事件
- 基础胜负判定

不进当前层的功能：

- 抛射
- 墙体破坏
- 搜寻、巡航、保卫等复杂 AI
- 玩家拖拽选点施法
- 高级脚本战斗编排

3. 战斗输入

建议世界层传入 `BattleRequest`：

- `battle_id`
- `event_instance_id`
- `map_id`
- `hero_snapshot`
- `ally_snapshot`
- `equipped_relic_modifiers`
- `run_modifiers`

4. 战斗输出

建议战斗层回传 `BattleResult`：

- `status`
- `victory`
- `defeat_reason`
- `casualties`
- `reward_package`
- `completed_objectives`

- `spawned_story_flags`
- `spawned_unlock_flags`
- `map_effects`

世界层不应直接读取战斗内部对象，而应只消费 `BattleResult`。

5. 子系统分工

`BattleDirector`

- 驱动整场战斗生命周期
- 协调单位系统、AI、战斗内事件和胜负判定

`UnitController`

- 维护单位数值和状态
- 处理伤害、死亡、受控状态

`AISystem`

- 处理基础索敌、追杀和攻击触发

`CombatSystem`

- 处理攻击频率、攻击距离、命中和伤害

`ProjectileSystem`

- 当前只支持基础平射

`BattleEventSystem`

- 处理触发、频率、通知、响应、结束

6. 与世界层的接口要求

- 战斗结束后必须只回传结构化结果
- 奖励不要在战斗层直接写入永久仓库
- 剧情标记和解锁标记可在结果中返回，由世界层统一应用
- 战斗失败是否导致死亡，由世界层结合事件规则解释

7. 战斗层验收标准

- 模板战斗能被事件系统拉起
- 单位会正确生成、移动、索敌、攻击、死亡
- 战斗内事件能触发和结束
- 战斗结束后能返回标准 `BattleResult`

==========
SOURCE: docs/agent/40-inventory-progression.md
==========

L2C Inventory Progression

1. 本层职责

本层负责区分“本局拿到的东西”和“真正带出地图的东西”。

它负责：

- 局内临时战利品
- 局外永久仓库
- 藏品装备
- 剧情标记和解锁标记的持久化
- 撤离成功与死亡失败的不同结算

2. 资源分层

局内临时层

包含：

- 本局获得的物资
- 本局获得的货币
- 本局获得的藏品
- 本局获得的临时 buff

特点：

- 挂在 `RunState`
- 只有撤离成功才真正转存

局外永久层

包含：

- 永久仓库物品
- 永久货币
- 永久藏品
- 已完成任务
- `story_flags`
- `unlock_flags`

特点：

- 挂在 `ProgressionState`
- 跨局保留

3. 死亡与撤离规则

撤离成功

- 本局临时物资、货币、藏品进入永久存档
- 本局完成任务写入已完成列表
- 本局新增 `story_flags` 和 `unlock_flags` 保留

死亡失败

- 本局临时物资、货币、藏品丢失
- 本局新增 `story_flags` 和 `unlock_flags` 保留
- 任务完成与否按任务配置决定是否保留

4. 藏品与装备

装备规则

- 仅 `type = 10` 的物品可装备
- 装备来源是永久仓库
- 开局时生成装备快照
- 进入战斗事件时，将装备快照转换为战斗修正

当前阶段不做

- 复杂格子扩展
- 复杂负荷模拟
- 背包拼图

5. 解锁与剧情

`story_flags` 和 `unlock_flags` 不能依附在 UI 层。

它们必须：

- 存在于 `ProgressionState`
- 可驱动后续任务和固定事件线
- 可驱动 UI 页签和系统开放

首批标准化开关：

- `narrative_events_enabled`
- `equipment_tab_enabled`
- `special_item_event_enabled`

6. 本层验收标准

- 局内奖励先进入临时容器
- 撤离后能正确转入永久仓库
- 死亡后临时资源被清空
- 藏品只能从永久仓库装备
- 解锁和剧情状态能影响下一局内容

==========
SOURCE: docs/agent/50-data-contracts.md
==========

L3 Data Contracts

1. 数据来源原则

- 策划源头是三个 Excel
- 运行时只读 JSON
- JSON 是唯一运行时真相

不要在游戏运行时直接解析 Excel。

2. 顶层内容文件建议

- `items.json`
- `loot_tables.json`
- `maps.json`
- `events.json`
- `tasks.json`
- `units.json`
- `battles.json`
- `battle_events.json`
- `progression_defaults.json`

3. 关键接口修正

事件定义不要只有一种“类型”，至少要拆成两层：

`spawn_mode`

- `random_slot`
- `fixed_line`

`resolution_type`

- `battle`
- `random`
- `narrative`

这是本项目当前最重要的数据建模修正。

4. `MapDef`

建议最小字段：

```
{
  "id": "map_world_1_bridge",
  "name": "世界一1地图1大桥附近",
  "random_slot_anchors": [
    { "id": "slot_a", "position": [120, 180] },
    { "id": "slot_b", "position": [260, 160] },
    { "id": "slot_c", "position": [380, 220] },
    { "id": "slot_d", "position": [150, 320] }
  ],
  "random_slot_count_range": [4, 6],
  "base_stats": {
    "development": 0,
    "public_reputation": 0,
    "government_reputation": 0,
    "demon_activity": 0,
    "fear": 0,
    "population": 0
```

```
  }
}
```

说明：

- `random_slot_anchors` 定义随机事件可出现的固定地图位置
- `random_slot_count_range` 用于控制每回合显示数量

5. `EventDef`

建议最小字段：

```
{
  "id": "event_trace_demon",
  "title": "恶魔的行踪",
  "spawn_mode": "random_slot",
  "resolution_type": "battle",
  "conditions": ["turn > 2"],
  "weight": 10,
  "map_ids": ["map_world_1_bridge"],
  "slot_tags": [],
  "line_id": "",
  "persistence": "refreshable",
  "battle_id": "battle_trace_demon",
  "random_resolution": null,
  "narrative_resolution": null,
  "reward_package": {},
  "on_success": [],
  "on_failure": []
}
```

说明：

- `spawn_mode = random_slot` 时，`persistence` 通常为 `refreshable`
- `spawn_mode = fixed_line` 时，`persistence` 通常为 `until_resolved`

6. `TaskDef`

建议最小字段：

```
{
  "id": "task_trace_demon",
  "name": "恶魔的行踪",
  "task_type": "search",
  "event_ref": "event_trace_demon",
  "pool_type": "probability",
  "entry_conditions": ["turn > 2"],
  "weight_rules": [
    { "if": "map.demon_activity > 5", "add": 5 }
  ],
  "reward_preview": [],
  "story_flags_on_complete": ["story_trace_demon_seen"],
  "unlock_flags_on_complete": [],
  "map_effects_on_complete": []
}
```

7. `RewardPackage`

```
{
  "currencies": [],
  "items": [],
  "relics": [],
  "buffs": [],
  "loot_tables": [],
  "story_flags": [],
  "unlock_flags": [],
  "world_effects": []
}
```

规则:

- 物资类奖励先进入 `RunState`
- `story_flags` 和 `unlock_flags` 直接落到 `ProgressionState`

8. `RunState`

```
{
  "run_id": "uuid",
  "map_id": "map_world_1_bridge",
  "turn": 1,
  "danger_level": 0,
  "board_state": {
    "random_slots": [],
    "fixed_events": [],
    "selected_event_id": ""
  },
  "map_stats": {},
  "temporary_inventory": [],
  "temporary_currencies": [],
  "temporary_relics": [],
  "completed_tasks": [],
  "can_extract": false,
  "is_dead": false,
  "is_extracted": false
}
```

9. `ExtractionResult`

```
{
  "status": "extracted",
  "saved_items": [],
  "saved_currencies": [],
  "saved_relics": [],
  "lost_items": [],
  "completed_tasks": [],
  "story_flags_applied": [],
  "unlock_flags_applied": []
}
```

10. 数据层验收标准

- 能区分随机事件与固定事件

- 能区分事件出现方式与事件结算方式
- 能表达随机事件固定位置刷新
- 能表达固定事件跨回合保留
- 能表达临时收益与永久收益分层

==========
SOURCE: docs/agent/90-task-packets.md
==========

Task Packets

1. 使用规则

给 AI 派任务时，不要直接粘贴整个长文档。请改用下面的最小上下文包。

每个任务包都应包含：

- 任务目标
- 需要加载的层
- 明确不在本次任务里的内容
- 验收标准

2. Packet A：工程底座

适用：

- 初始化 Godot 工程结构
- 建立 Autoload
- 导入 JSON

附带层：

- `L0`
- `L1`
- `L3`

任务描述模板：

目标：搭建项目工程底座，不实现完整玩法。
加载层：L0 + L1 + L3。
只做：Autoload、基础目录结构、JSON 加载和校验。
不要做：世界事件生成、战斗、UI 细化。
验收：项目可启动，JSON 可加载，坏引用会报错。

3. Packet B：世界层 / 事件刷新

适用：

- 随机事件刷新
- 固定事件线
- 任务池
- 叙事触发

附带层：

- `L0`
- `L1`
- `L2A`
- `L3`

任务描述模板：

目标：实现地图回合事件板。
加载层：L0 + L1 + L2A + L3。
只做：4-6 个随机事件的生成、固定事件保留、选择一个事件进入结算。
不要做：战斗内部逻辑、永久仓库存档。
验收：随机事件刷新在固定地图位置，固定事件不会被刷新覆盖。

4. Packet C：战斗执行器

适用：

- 模板战斗
- 单位、AI、战斗内事件

附带层：

- `L0`
- `L1`
- `L2B`
- `L3`

任务描述模板：

目标：实现 battle 类型事件的执行器。
加载层：L0 + L1 + L2B + L3。
只做：MVP 自动战斗、标准 BattleResult 返回。
不要做：随机事件刷新、永久奖励入库。
验收：世界层可以拉起战斗并收到结果。

5. Packet D：背包 / 撤离 / 存档

适用：

- 临时战利品
- 撤离结算
- 死亡清算
- 装备与永久仓库

附带层：

- `L0`
- `L1`
- `L2C`
- `L3`

任务描述模板：

目标：实现局内临时收益和局外永久收益分层。
加载层：L0 + L1 + L2C + L3。
只做：临时背包、撤离转存、死亡清空、装备读取。
不要做：复杂掉落池、家园或图鉴。
验收：撤离成功后资源进入永久存档，死亡后临时资源丢失。

6. Packet E：数据结构调整

适用：

- 新增 JSON 字段
- 修改运行时状态
- 改跨系统接口

附带层：

- `L0`
- 相关 `L2`
- `L3`

任务描述模板：

目标：调整数据契约，不扩展无关玩法。
加载层：L0 + 对应 L2 + L3。
只做：本次变更需要的 schema、加载器、校验器和调用方同步。
不要做：额外 UI 重构或新玩法扩展。
验收：旧数据迁移清晰，新结构能被上下游正确消费。